途正英语测评引擎 v3 — 后端改造技术文档

版本：v3.0（智能预判引擎） 编写：火鹰科技 日期：2026年4月8日

一、架构总览

v3引擎将原来的 `evaluate` 接口拆分为两个独立阶段：

阶段一（做题过程中）： 轻量提交 `submit-lite`，不调用LLM，毫秒级响应，用规则引擎预判并返回下一题。

阶段二（测评结束后）： 异步精评 `async-evaluate`，后台逐题调用LLM精确评分，生成完整报告。前端通过轮询 `report-status` 接口获取报告生成进度。

```
做题过程：
  前端 → submit-lite → 后端规则引擎（毫秒级）→ 返回下一题 → 前端

测评结束后：
  后端异步任务 → Whisper转写 → LLM逐题评分 → 综合定级 → 生成报告
  前端 → report-status（轮询）→ 报告完成 → 展示结果
```

二、新增接口规范

2.1 轻量提交接口 `POST /api/v1/test/submit-lite`

这是做题过程中的核心接口，替代原来的 `evaluate`。不调用任何AI服务，纯规则计算，响应时间应控制在 **200ms以内**。

请求参数：

字段	类型	必填	说明
sessionId / session_id	string	是	测评会话ID
questionId / question_id	string	是	当前题目ID
audioUrl / audio_url	string	否	录音OSS地址（前端上传后获得）
recognizedText / recognized_text	string	是	前端语音识别文本（可为空字符串）
duration	number	否	录音时长（毫秒）
responseDelay / response_delay	number	否	响应延迟：从题目音频播放结束到用户开始录音的时间间隔（毫秒）
questionText / question_text	string	否	题目原文（前端传回，用于后续异步评分时的上下文）
skipped	boolean	否	是否为跳过（true表示用户主动跳过此题）

响应格式（status=continue）：

```json
{
  "code": 200,
  "data": {
    "status": "continue",
    "sessionId": "xxx",
    "currentSubLevel": "G3",
    "currentMajorLevel": 1,
    "questionIndex": 5,
    "totalAnswered": 4,
    "levelUp": true,
    "levelUpMessage": "升级到G3",
    "skipLevel": false,
    "question": {
      "questionId": "q_xxx",
      "audioUrl": "https://...",
      "questionText": "What do you usually do after school?",
      "subLevel": "G3"
    },
    "prediction": {
      "passed": true,
      "confidence": "high",
      "signals": {
        "textLength": 18,
        "duration": 6500,
        "responseDelay": 1200,
        "sentenceComplete": true
      }
    }
  }
}
```

响应格式（status=finished）：

```
{
  "code": 200,
  "data": {
    "status": "finished",
    "sessionId": "xxx",
    "totalAnswered": 12,
    "question": null,
    "pendingReport": true,
    "estimatedWaitSeconds": 45,
    "preliminaryLevel": {
      "majorLevel": 1,
      "majorLevelName": "一级",
      "majorLevelLabel": "小学水平"
    }
  }
}
```

`pendingReport: true` 表示报告正在异步生成中，前端需要轮询 `report-status` 接口。`preliminaryLevel` 是基于预判的初步等级，供前端在等待期间先展示一个大致结果。`estimatedWaitSeconds` 是预估的报告生成等待时间。

2.2 报告状态查询接口 `GET /api/v1/test/report-status/:sessionId`

前端轮询此接口，检查异步评分是否完成。

响应格式（生成中）：

```
{
  "code": 200,
  "data": {
    "status": "processing",
    "progress": 60,
    "progressText": "正在分析第8题/共12题",
    "estimatedRemainingSeconds": 20
  }
}
```

响应格式（已完成）：

```
{
  "code": 200,
  "data": {
    "status": "completed",
    "progress": 100
  }
}
```

响应格式（失败）：

```
{
  "code": 200,
  "data": {
    "status": "failed",
    "error": "AI评分服务暂时不可用",
    "canRetry": true
  }
}
```

前端收到 `status: "completed"` 后，调用现有的 `GET /api/v1/test/report/:sessionId` 接口获取完整报告。

2.3 触发重新生成报告 `POST /api/v1/test/report-retry/:sessionId`

当报告生成失败时，前端可调用此接口触发重新生成。

响应格式：

```
{
  "code": 200,
  "data": {
    "status": "processing",
    "message": "已重新开始生成报告"
  }
}
```

2.4 现有接口保留

以下现有接口保持不变，无需修改：

- `POST /api/v1/test/start` — 创建测评会话，返回第一题
- `POST /api/v1/test/upload-audio` — 上传录音文件
- `GET /api/v1/test/report/:sessionId` — 获取完整报告（异步评分完成后可用）
- `GET /api/v1/test/result/:sessionId` — 获取结果摘要（降级兼容）
- `POST /api/v1/test/confirm-level` — 确认最终评级
- `GET /api/v1/test/user-level-status` — 查询用户分级确认状态

原有的 `POST /api/v1/test/evaluate` 接口建议保留但标记为deprecated，前端将不再调用。

三、预判算法详细规范

3.1 预判信号采集

每道题提交后，后端从以下维度提取预判信号：

信号维度	字段来源	计算方式
文本长度（wordCount）	recognizedText	按空格分词后的单词数
录音时长（durationSec）	duration	毫秒转秒
响应延迟（responseDelaySec）	responseDelay	毫秒转秒，题目播完到开始录音的间隔
是否跳过（skipped）	skipped字段 或 recognizedText为空且duration<1000	布尔值
句子完整度（sentenceComplete）	recognizedText	正则检测是否包含主谓结构（至少有动词）
词汇丰富度（vocabRich）	recognizedText	是否包含非基础200词表之外的单词

3.2 单题预判评分规则

根据采集到的信号，计算该题的**预判分**（0-100），用于驱动出题决策：

```
预判分计算规则：

如果 skipped == true：
    预判分 = 0

否则：
    基础分 = 0

    // 文本长度得分（权重40%）
    if wordCount >= 15:  基础分 += 40
    elif wordCount >= 8:  基础分 += 30
    elif wordCount >= 4:  基础分 += 15
    elif wordCount >= 1:  基础分 += 5
    else:                 基础分 += 0

    // 录音时长得分（权重25%）
    if 3 <= durationSec <= 15:  基础分 += 25
    elif 2 <= durationSec < 3:  基础分 += 15
    elif 15 < durationSec <= 25: 基础分 += 10  // 太长可能在犹豫
    else:                        基础分 += 0

    // 响应速度得分（权重15%）
    if responseDelaySec <= 3:   基础分 += 15  // 快速反应
    elif responseDelaySec <= 6: 基础分 += 10
    elif responseDelaySec <= 10: 基础分 += 5
    else:                       基础分 += 0   // 犹豫太久

    // 句子完整度得分（权重10%）
    if sentenceComplete:  基础分 += 10

    // 词汇丰富度得分（权重10%）
    if vocabRich:  基础分 += 10

    预判分 = 基础分
```

预判分的通过门槛为 **50分**（而非AI精评的60分，因为预判本身就是粗略的，门槛设低一点避免误判）。

3.3 自适应出题决策算法

后端维护每个session的答题状态，包括：当前小级、当前小级已答题数、当前小级的预判分列表、连续高质量回答计数。

```
出题决策算法（每次submit-lite调用后执行）：

输入：当前小级(subLevel)、该小级已答题数(count)、该小级预判分列表(scores)
输出：动作(action) = "next_in_level" | "level_up" | "skip_level" | "finish"

// ====== 快速结束检测 ======
// 连续3题预判分为0（跳过或完全没答）→ 立即结束
if 最近连续3题预判分 == 0:
    action = "finish"
    return

// 连续4题预判分 < 30（极低质量回答）→ 立即结束
if 最近连续4题预判分 < 30:
    action = "finish"
    return

// ====== 快速跳级检测 ======
// 连续2题预判分 >= 85（高质量信号）→ 跳级
if count >= 2 AND 最近2题预判分都 >= 85:
    // 跳过2个小级（如PRE1 → G1，G1 → G4）
    action = "skip_level"
    skipCount = 2
    return

// ====== 正常推进逻辑 ======
if count >= 2 AND 最近2题预判分都 >= 50:
    // 连续2题通过 → 升级到下一个小级
    action = "level_up"
    return

if count >= 3 AND 该小级平均预判分 < 50:
    // 3题平均分不及格 → 结束测评
    action = "finish"
    return

if count >= 4:
    // 已答4题，无论如何进入下一步
    if 该小级平均预判分 >= 50:
        action = "level_up"
    else:
        action = "finish"
    return
```

```
// 还没答够，继续在当前小级出题
action = "next_in_level"
```

3.4 小级路径与跳级映射

正常路径（从低到高）：

```
PRE1 → PRE2 → G1 → G2 → G3 → G4 → G5 → G6 → G7 → G8 → G9 → G10 → G11 → G12 →
IELTS4 → IELTS5 → IELTS6 → IELTS7 → IELTS8 → IELTS9
```

跳级映射（skipCount=2时，从当前小级跳到目标小级）：

当前小级	跳2级后	说明
PRE1	G1	跳过PRE2，直接进入一级
PRE2	G2	跳过G1
G1	G3	在一级内跳
G2	G4	在一级内跳
G3	G5	在一级内跳
G4	G6	在一级内跳
G5	G7	跳入二级
G6	G8	跳入二级
…	…	以此类推

如果跳级后已经超出最高小级（IELTS9），则直接结束测评。

3.5 全局保护规则

为确保定级的基本准确性，设置以下全局保护：

- **最少答题数：5题。** 无论预判结果如何，至少要答5题才允许结束。
- **最多答题数：30题。** 达到30题后强制结束，防止无限循环。

- **每个大级至少经过1个小级。** 即使跳级，也要确保在每个大级至少答过题目，才能判定该大级的水平。

四、异步精评流程

4.1 触发时机

当 `submit-lite` 返回 `status: "finished"` 时，后端立即启动异步精评任务（可用消息队列或后台Worker）。

4.2 精评步骤

对该session的所有已答题目，按顺序执行：

步骤一：语音转写（Whisper）

对每道题的录音文件调用Whisper API进行精确转写。如果前端已提供recognizedText且长度大于5个单词，可以跳过Whisper转写直接使用前端文本（节省时间和成本）。

步骤二：逐题LLM评分

对每道题调用LLM，输入题目原文和转写文本，输出四维评分：

- pronunciation（发音准确度，0-100）
- grammar（语法运用，0-100）
- vocabulary（词汇量，0-100）
- fluency（口语流利度，0-100）

以及单题综合分（四维加权平均）和反馈文本。

步骤三：综合定级

根据所有题目的精确评分，计算最终等级：

```
综合定级算法：

1. 计算每个小级的平均精确分
2. 找到最高的"通过小级"（平均分 >= 60 × 保守系数的倒数）
3. 将最高通过小级映射到大级
4. 应用保守系数：最终得分 = 原始综合分 × conservativeRatio
5. 根据最终得分确定大级别（0/1/2/3）

大级别分数区间（应用保守系数后）：
  0级（零级·学前水平）：最终得分 < 40
  1级（一级·小学水平）：40 <= 最终得分 < 60
  2级（二级·中学水平）：60 <= 最终得分 < 75
  3级（三级·雅思水平）：最终得分 >= 75
```

步骤四：生成报告

调用LLM生成综合评价文本，包括：summary（总评）、strengths（优势）、weaknesses（薄弱环节）、recommendation（学习建议）。

4.3 保守定级系数

保守系数 `conservativeRatio` 存储在系统配置表中，默认值为 **0.85**，后台管理界面可调整。

```sql
-- 系统配置表
INSERT INTO system_config (config_key, config_value, description)
VALUES ('conservative_ratio', '0.85', '定级保守系数，范围0.75-1.00，越低越保守');
```

应用方式：

```
最终综合得分 = 原始AI评分的加权平均 × conservativeRatio
```

4.4 进度上报

异步任务在处理过程中，更新session的报告生成进度（存Redis或数据库），供 `report-status` 接口查询：

```
进度计算：
  progress = (已完成评分的题目数 / 总题目数) × 90 + (报告生成完成 ? 10 : 0)

  例如12题的测评：
    第1题评完 → progress = 7.5%
    第6题评完 → progress = 45%
    第12题评完 → progress = 90%
    报告文本生成完毕 → progress = 100%
```

五、数据库变更

5.1 答题记录表新增字段

```sql
ALTER TABLE test_answers ADD COLUMN response_delay INT DEFAULT NULL COMMENT
'响应延迟（毫秒）';
ALTER TABLE test_answers ADD COLUMN prediction_score INT DEFAULT NULL
COMMENT '预判分（0-100）';
ALTER TABLE test_answers ADD COLUMN prediction_passed TINYINT DEFAULT NULL
COMMENT '预判是否通过（0/1）';
ALTER TABLE test_answers ADD COLUMN ai_score_status ENUM('pending',
'processing', 'completed', 'failed') DEFAULT 'pending' COMMENT 'AI精评状态';
```

5.2 测评会话表新增字段

```sql
ALTER TABLE test_sessions ADD COLUMN report_status ENUM('none',
'processing', 'completed', 'failed') DEFAULT 'none' COMMENT '报告生成状态';
ALTER TABLE test_sessions ADD COLUMN report_progress INT DEFAULT 0 COMMENT
'报告生成进度（0-100）';
ALTER TABLE test_sessions ADD COLUMN preliminary_level INT DEFAULT NULL
COMMENT '预判初步等级（0/1/2/3）';
ALTER TABLE test_sessions ADD COLUMN conservative_ratio DECIMAL(3,2) DEFAULT
0.85 COMMENT '本次测评使用的保守系数';
ALTER TABLE test_sessions ADD COLUMN report_started_at DATETIME DEFAULT NULL
COMMENT '报告开始生成时间';
ALTER TABLE test_sessions ADD COLUMN report_completed_at DATETIME DEFAULT
NULL COMMENT '报告生成完成时间';
```

5.3 系统配置表

```sql
CREATE TABLE IF NOT EXISTS system_config (
  id INT AUTO_INCREMENT PRIMARY KEY,
  config_key VARCHAR(100) UNIQUE NOT NULL,
  config_value TEXT NOT NULL,
  description VARCHAR(500),
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

INSERT INTO system_config (config_key, config_value, description) VALUES
('conservative_ratio', '0.85', '定级保守系数，范围0.75-1.00'),
('min_questions', '5', '最少答题数'),
('max_questions', '30', '最多答题数'),
('skip_level_threshold', '85', '快速跳级的预判分门槛'),
('quick_finish_consecutive_zeros', '3', '连续N题零分则快速结束'),
('prediction_pass_threshold', '50', '预判通过门槛分');
```

六、前端改造要点（已由火鹰科技完成）

前端的改造已经同步进行，主要变更包括：

1. **api.js** 新增 `submitLite()`、`getReportStatus()`、`retryReport()` 三个接口封装
2. **home.js** 登录成功后立即在后台调用 `startTest` 预加载第一题数据，存入全局缓存
3. **test.js** 将 `evaluateAnswer` 替换为 `submitLite`，去掉 evaluating 等待阶段，录音结束后立即播放下一题；新增 `responseDelay` 的记录逻辑
4. **result.js** 新增轮询机制，每3秒调用 `report-status` 检查报告生成进度，完成后自动加载完整报告

前端代码已兼容新旧两套接口：如果后端尚未部署 `submit-lite`，前端会自动降级到原有的 `evaluate` 接口。

七、部署与灰度建议

1. **第一步**：部署数据库变更（新增字段和配置表）

2. **第二步**：部署 `submit-lite` 和 `report-status` 接口，保留原有 `evaluate` 接口不变
3. **第三步**：前端发布新版本，开始使用 `submit-lite`
4. **第四步**：观察1-2周运营数据，对比预判准确率和AI精评结果的偏差
5. **第五步**：根据数据微调预判算法参数和保守系数

火鹰科技 · 广州火鹰信息科技有限公司 技术咨询：ceo@figo.cn